Week 47: Boosting

From bagging to boosting – AdaBoost by hand and from scratch, steepest descent in function space, gradient boosting with `scikit-learn`, and XGBoost

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 47

Monday lecture (2×45 min): Chapter 7, Sections 7.11–7.16.

1. **Repetition**: why ensembles at all – the variance of a tree, Eq. (7.16), bagging and random forests, and what they cannot do.
2. **Boosting from a bird's eye view** (7.11): additive models, forward stagewise fitting, a toy example worked out by hand.
3. **AdaBoost** (7.12): the algorithm, a first round by hand, where the strange formulas come from (the exponential loss), what the classifier estimates, margins.
4. **Our own AdaBoost against** `scikit-learn`: weighted trees, the class of Section 7.12, the moons.
5. **Steepest descent in function space** (7.13): the gradient of a loss with respect to the predictions, why the literal algorithm is useless, and how a tree repairs it.
6. **Gradient boosting** with `scikit-learn`: regression, shrinkage against the number of trees, classification with the logistic loss.
7. **XGBoost** (7.14): second-order objective, the closed-form leaf weights, the gain, λ and γ, the cancer data.
8. **Summary** and the plan for weeks 47–48.

Tuesday (flipped classroom, the last one): your Project-3 proposal presentations, 3–5 slides and 3–5 minutes per group. The slides are the week-47 exercise, due Friday November 20.
Reading: Sections 7.11–7.16; Hastie, Tibshirani and Friedman Chapter 10; Chen and Guestrin, *XGBoost: A Scalable Tree Boosting System* (2016). Code: `week47.ipynb`, `make_week47_figures.py`.

Where we were: a single tree is a high-variance estimator

Last week we grew a tree by recursive binary splitting and found it **readable**, **fast**, **scale-free** – and **unstable**. The reason was quantitative (week 46, "why a tree has high variance"):

- a leaf with n_m points predicts their mean, with variance σ^2/n_m; a fully grown tree has $n_m = 1$ and reproduces the noise, expected test error $2\sigma^2$ – twice the irreducible error;
- a shallow tree has small variance but large bias: a piecewise constant with few pieces;
- on top of that the *partition itself* is random – a slightly different training set moves the cutpoints, and by the law of total variance that adds a second term.

On the noisy Runge function: depth-4 tree 0.0181, full tree 0.0214, a degree-10 polynomial 0.0133, noise floor 0.0100.

The two ways out

Average many high-variance, low-bias trees so the noise cancels (bagging, random forests – last week), or **add up** many low-variance, high-bias trees so the bias cancels (boosting – today). Both build

$$f(\mathrm{x}) = \sum_{m=1}^{M} \beta_m \, T_m(\mathrm{x}),$$

and the whole difference is in how the T_m and the β_m are chosen: independently and in parallel, or one after the other, each correcting the last.

Repetition 1: the variance of an average, Eq. (7.16)

B estimators $f_1, \ldots, f_B$, each with variance σ^2, pairwise correlation ρ. Their average $\bar{f} = \frac{1}{B}\sum_b f_b$ has

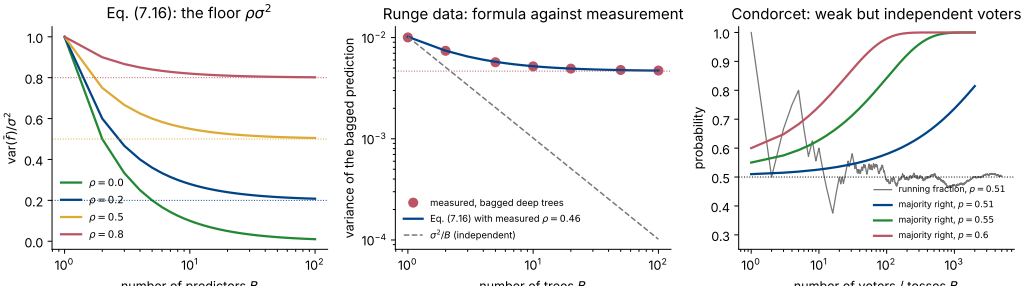
$$\operatorname{var}(\bar{f}) = \frac{1}{B^2}\Big[\sum_b \operatorname{var}(f_b) + \sum_{b \neq b'} \operatorname{cov}(f_b, f_{b'})\Big]$$

$$= \frac{\sigma^2}{B} + \frac{B(B-1)}{B^2}\rho\sigma^2 = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

- independent members ($\rho = 0$): variance σ^2/B, Eq. (7.14);
- correlated members: the second term dies with B, the **floor** $\rho\sigma^2$ **does not**;
- averaging never changes the **expectation**: $\mathbb{E}\bar{f} = \mathbb{E}f_b$. The bias of the members is the bias of the average.

Bagging (Section 7.9): B bootstrap samples, one deep tree on each, average (regression) or majority vote (classification). The out-of-bag points of each tree give a free test set, Eq. (7.17).
Random forests (Section 7.10): the same, but at every split only $m \approx \sqrt{p}$ random features are candidates, Eq. (7.18). Bagged trees of the same data all split on the same strong feature first and are correlated; restricting the candidates **lowers** ρ and therefore the floor.
Measured last week on the Runge data: $\sigma^2 = 0.0102$ for one tree, $\rho = 0.46$ between two bagged trees – averaging a hundred of them removes about half the variance and then stops. On the cancer data ρ fell from 0.81 (bagging) to 0.78 (forest, $m = 5$).

Repetition 2: Eq. (7.16) measured, and the ensembles on the moons



Left: $\mathrm{var}(\bar{f})/\sigma^2$ against B for several ρ; dotted lines are the floors. **Middle**: 200 independent training sets of the noisy Runge function, 100 bagged trees on each – the measured variance of the average follows the formula with the measured σ^2 and ρ and no fitting. **Right**: Condorcet's jury – B independent voters each right with probability $p > 1/2$ become a strong majority. The italics are the assumption that bagging only approximately satisfies, and that boosting will replace by a different mechanism altogether.

Repetition 3: what we got on the data sets, and what we did not get

Moons ($n = 500$, noise 0.3, 375/125 split): single tree 0.824, bagging 0.907 (0.923 in `scikit-learn`), random forest 0.909 (0.915). **Breast cancer** (569×30, 398/171 split): tree of depth 3 0.918, full tree 0.930, bagging 0.942, random forest 0.953 – **logistic regression** 0.965. The islands around single points were averaged away, the boundary became smooth and came with a graded confidence (the fraction of votes). The gain was complete by $B \approx 50$.

What averaging cannot do

- It cannot remove the **bias** of its members. A forest of stumps (depth 1) is still additive in the features: no averaging makes it learn an interaction.
- It cannot do better than **the floor** $\rho\sigma^2$.
- It treats every training point the same at every step; a point every tree gets wrong stays wrong.

Boosting fixes all three by **building the members sequentially**: each new one is fitted to what the previous ones got wrong. The members can then be very simple – *weak learners* – and the bias is reduced step by step.

Pause and think 1: can bagging ever beat the bias of its members?

Question

We bag $B = 10^6$ decision *stumps* (one split each) on data with a target $y = x_1 x_2 + \epsilon$, $x_1, x_2 \in \{-1, +1\}$. Does the bagged prediction converge to the truth as $B \to \infty$? What does the same question look like for boosting?

Pause and think 1: can bagging ever beat the bias of its members?

Question

We bag $B = 10^6$ decision *stumps* (one split each) on data with a target $y = x_1 x_2 + \epsilon$, $x_1, x_2 \in \{-1, +1\}$. Does the bagged prediction converge to the truth as $B \to \infty$? What does the same question look like for boosting?

Answer

No. A stump is a function of one feature, so every member is of the form $g(x_1)$ or $h(x_2)$, and so is any average of them – an additive function $a(x_1) + b(x_2)$. But $x_1 x_2$ is orthogonal to every additive function on this domain (sum over the four corners), so the best additive approximation is the constant 0: the bagged prediction converges, with probability one, to a function with **zero correlation with the target**. Averaging removes the variance $\sigma^2(1 - \rho)/B$, not the bias. Boosting stumps gives the same additive class, so it fails here too – but boosting trees of depth 2 can represent $x_1 x_2$ exactly: the depth of the weak learner sets the **interaction order** of the model, one of the few knobs that matter.

Boosting from a bird's eye view: additive models fitted forward

Boosting fits an **additive expansion** in a set of simple basis functions $b(\mathrm{x};\gamma)$ – stumps, small trees, splines, anything with a few parameters γ:

$$f_M(\mathrm{x}) = \sum_{m=1}^{M} \beta_m\, b(\mathrm{x};\gamma_m). \tag{7.19}$$

Fitting all (β_m, γ_m) at once,

$$\min_{\{\beta_m,\gamma_m\}} \sum_{i=1}^{n} L\Big(y_i, \sum_m \beta_m\, b(\mathrm{x}_i;\gamma_m)\Big),$$

is hopeless for trees. **Forward stagewise** fitting instead adds one term at a time and never revisits the old ones:

$$(\beta_m,\gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1}^{n} L\big(y_i,\ f_{m-1}(\mathrm{x}_i) + \beta\, b(\mathrm{x}_i;\gamma)\big),$$
$$f_m(\mathrm{x}) = f_{m-1}(\mathrm{x}) + \beta_m b(\mathrm{x};\gamma_m), \qquad f_0 = 0. \tag{7.20}$$

Three ideas in one line

1. **Weak learners**: b is simple, slightly better than guessing.
2. **Sequential**: term m sees the residual of terms $1..m-1$, so it concentrates on what is still wrong – in AdaBoost through *weights* on the points, in gradient boosting through *pseudo-residuals*.
3. **Greedy**: each (β_m, γ_m) is optimal given the past and frozen afterwards. Slow, but it works – and the slowness turns out to be a form of regularization.

The whole lecture is Eq. (7.20) for three choices of the loss L and of the weak learner b: the exponential loss and stumps (AdaBoost), any differentiable loss and regression trees (gradient boosting), and a second-order expansion of any loss with regularized trees (XGBoost).

A toy case worked out: $b(x;\gamma) = 1 + \gamma x$ and the squared error

Take one feature, $L(y, f) = (y - f)^2$ and the two-parameter basis function $b(x;\gamma) = 1 + \gamma x$. One stage of Eq. (7.20) with residual vector $\mathrm{r} = \mathrm{y} - f_{m-1}(\mathrm{x})$ is

$$\min_{\beta,\gamma} \; \|\mathrm{r} - \beta(\mathrm{e} + \gamma \mathrm{x})\|^2, \qquad \mathrm{e} = (1, \ldots, 1)^T.$$

Write $\mathrm{w} = \mathrm{e} + \gamma \mathrm{x}$. For fixed γ the problem in β is ordinary least squares with one column,

$$\beta = \frac{\mathrm{w}^T \mathrm{r}}{\mathrm{w}^T \mathrm{w}}.$$

For fixed β, $\partial/\partial\gamma$ of the sum of squares gives $\mathrm{x}^T(\mathrm{r} - \beta\mathrm{e} - \beta\gamma\mathrm{x}) = 0$, that is

$$\gamma = \frac{\mathrm{x}^T \mathrm{r} - \beta\, \mathrm{x}^T \mathrm{e}}{\beta\, \mathrm{x}^T \mathrm{x}}.$$

Alternate the two until they stop moving: an inner **iterative fit** of the basis function, then the outer stagewise step $f_m = f_{m-1} + \beta(1 + \gamma x)$, then the next residual.

What to take from the toy

- A basis function with a nonlinear parameter (γ inside b) needs an inner optimization; for a tree, γ is the set of splits and leaf values and the inner optimization is **CART itself**.
- Only the **residual** enters stage m. The squared error makes this literal; other losses replace the residual by a generalized one (the negative gradient, Section 7.13).
- Nothing forces β_m to be the exact minimizer – shrinking it, $f_m = f_{m-1} + \nu\beta_m b$, $0 < \nu < 1$, costs more stages and buys stability. All libraries do this.

Weak learners

A **weak learner** for two classes is any rule with error rate

$$\overline{\mathrm{err}} = \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq G(\mathrm{x}_i)) < \tfrac{1}{2}, \qquad (7.21)$$

strictly better than a coin – a **decision stump** (a tree with one split, so the classifier is a threshold on one feature) is the canonical example. Stumps alone are poor: on the moons a single stump gets 0.18 of the test points wrong. The claim of boosting (Schapire 1990, Freund and Schapire 1997) is that a weighted vote of weak learners,

$$G(\mathrm{x}) = \operatorname{sign}\Big(\sum_{m=1}^{M} \alpha_m G_m(\mathrm{x})\Big), \qquad (7.22)$$

can be made **arbitrarily strong** on the training data, provided each G_m is trained on a reweighted version of the data that emphasizes what $G_1, \ldots, G_{m-1}$ got wrong.

Contrast with Condorcet

Condorcet's jury needs *independent* voters, which bagging only approximates. Boosting makes no independence assumption: the voters are built to be **complementary**, each one competent exactly where the previous ones were not, and the weights α_m say how much each voter is trusted. We now derive the only weights for which this is a principled procedure.

Notation for the rest of the lecture

Labels $y_i \in \{-1, +1\}$ for AdaBoost (the sign trick needs it), $\{0, 1\}$ for the logistic loss, real for regression; $f(\mathrm{x})$ is the real-valued score, $G(\mathrm{x}) = \operatorname{sign} f(\mathrm{x})$ the class.

AdaBoost (Freund and Schapire 1997): the algorithm

Adaptive boosting, discrete version

1. Initialize the weights $w_i = 1/n$, $i = 1, \ldots, n$.
2. For $m = 1, \ldots, M$:
   (a) fit a classifier $G_m(\mathrm{x})$ to the training data using the weights w_i – a stump chosen by the *weighted* Gini index;
   (b) its weighted error
   $$\overline{\mathrm{err}}_m = \frac{\sum_i w_i I(y_i \neq G_m(\mathrm{x}_i))}{\sum_i w_i}; \qquad (7.26)$$
   (c) its vote $\alpha_m = \log \frac{1 - \overline{\mathrm{err}}_m}{\overline{\mathrm{err}}_m}$; (7.29)
   (d) update the weights
   $$w_i \leftarrow w_i \exp\left[\alpha_m I(y_i \neq G_m(\mathrm{x}_i))\right], \qquad (7.28)$$
   and renormalize them to sum to one.
3. Output $G(\mathrm{x}) = \operatorname{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(\mathrm{x})\right]$, Eq. (7.22).

Reading the formulas.

- $\overline{\mathrm{err}}_m < \frac{1}{2} \Rightarrow \alpha_m > 0$; $\overline{\mathrm{err}}_m \to 0 \Rightarrow \alpha_m \to \infty$ (a perfect learner decides alone); $\overline{\mathrm{err}}_m = \frac{1}{2} \Rightarrow \alpha_m = 0$ (a coin is ignored).
- Only the **misclassified** points are reweighted, all by the same factor $e^{\alpha_m} = (1 - \overline{\mathrm{err}}_m)/\overline{\mathrm{err}}_m$; after renormalization they carry exactly half the weight, so G_m is a coin on the new problem.
- The data never change, the **weights** do: the weak learner must accept sample weights – the only change to last week's tree.

AdaBoost by hand: ten points on a line

Points $x_i = i$, $i = 1, \ldots, 10$, labels $+,+,+,-,-,-,+,+,+,-$. No single threshold separates them; the best stump gets three wrong. Weak learner: a stump $x \le t$ with either orientation, t a midpoint.

Round 1, $w_i = 0.1$. The best stump is "$x \le 3.5 \Rightarrow +1$", wrong on points 7, 8, 9:

$$\overline{\mathrm{err}}_1 = 0.3, \qquad \alpha_1 = \log \frac{0.7}{0.3} = 0.847, \qquad e^{\alpha_1} = \tfrac{7}{3}.$$

Points 7, 8, 9 get weight $0.1 \cdot \frac{7}{3} = 0.233$, the others keep 0.1; total 1.4; after renormalization

$$w_{7,8,9} = \frac{0.233}{1.4} = 0.167, \qquad w_{\text{others}} = \frac{0.1}{1.4} = 0.0714.$$

Check: the three misclassified points now carry $3 \times 0.167 = 0.5$ – half the weight, as promised. The combined classifier is still G_1, training error 0.3.

Why half, in general

Before the update the misclassified points carry $\overline{\mathrm{err}}_m$ of the weight, the rest $1 - \overline{\mathrm{err}}_m$. Multiply the first group by $(1 - \overline{\mathrm{err}}_m)/\overline{\mathrm{err}}_m$: it now carries $1 - \overline{\mathrm{err}}_m$, the same as the second group. The factor in Eq. (7.28) is the *unique* one that makes G_m a coin on the reweighted data – the next learner cannot gain anything by repeating G_m, it must find something new.

AdaBoost by hand: rounds 2 and 3

Round 2, weights $(0.071)^{\times 6}$ on 1–6 and 10, 0.167 on 7–9. The stump "$x \le 9.5 \Rightarrow +1$" is wrong on 4, 5, 6 only (weight 3×0.0714):

$$\overline{\text{err}}_2 = 0.214, \quad \alpha_2 = \log \frac{0.786}{0.214} = 1.299.$$

New weights: 0.167 on 4–6, 0.106 on 7–9, 0.045 on 1–3 and 10.

Round 3, the stump "$x \le 6.5 \Rightarrow -1$" is wrong on 1, 2, 3, 10 (weight 4×0.045):

$$\overline{\text{err}}_3 = 0.182, \quad \alpha_3 = \log \frac{0.818}{0.182} = 1.504.$$

Combined: $f(x) = 0.847\, G_1 + 1.299\, G_2 + 1.504\, G_3$, with $G_1 = \text{sign}(3.5 - x)$, $G_2 = \text{sign}(9.5 - x)$, $G_3 = \text{sign}(x - 6.5)$:

x	2	5	8	10
$f(x)$	$+0.64$	-1.05	$+1.96$	-0.64

all four signs right – **training error zero** with three stumps, none of which gets fewer than three points wrong.

weights after round	1	2	3
points 1–3	0.071	0.045	0.125
points 4–6	0.071	0.167	0.102
points 7–9	0.167	0.106	0.065
point 10	0.071	0.045	0.125

What happened

Every round the weight moved to the points the *last* stump missed, and the next stump was forced to get *those* right, at the price of others. The votes α_m grew because each stump was fitted to a problem where the hard points dominated and did well on them. Three stumps, each a threshold, add up to a function with two sign changes – a function no single stump can represent. (`adaboost_by_hand()` in the notebook prints every number.)

Where AdaBoost comes from: the exponential loss

AdaBoost is forward stagewise fitting, Eq. (7.20), of the additive model $f(x) = \sum_m \beta_m G_m(x)$ with $G_m \in \{-1, +1\}$ and the **exponential loss**

$$L(y, f(x)) = \exp\big(-y\, f(x)\big). \tag{7.23}$$

Stage m must solve

$$(\beta_m, G_m) = \arg\min_{\beta, G} \sum_{i=1}^{n} \exp\big[-y_i \big(f_{m-1}(x_i) + \beta G(x_i)\big)\big].$$

The part that does not depend on (β, G) factors out:

$$= \arg\min_{\beta, G} \sum_{i=1}^{n} w_i^{(m)} e^{-\beta\, y_i G(x_i)}, \quad w_i^{(m)} = e^{-y_i f_{m-1}(x_i)}. \tag{7.24}$$

This is where the weights come from: $w_i^{(m)}$ is large where the current ensemble is wrong, and depends only on the past.

Since $y_i G(x_i) = \pm 1$, split the sum into the points G gets right and wrong,

$$\begin{aligned}\sum_i w_i^{(m)} e^{-\beta y_i G_i} &= e^{-\beta} \sum_{y_i = G_i} w_i^{(m)} + e^{\beta} \sum_{y_i \neq G_i} w_i^{(m)} \\ &= (e^{\beta} - e^{-\beta}) \sum_i w_i^{(m)} I(y_i \neq G_i) \\ &\quad + e^{-\beta} \sum_i w_i^{(m)}, \end{aligned} \tag{7.25}$$

with $G_i \equiv G(x_i)$.

Two consequences

For any $\beta > 0$ the first term is the only one that depends on G, so the best G_m is the one that **minimizes the weighted error**

$$G_m = \arg\min_G \sum_i w_i^{(m)} I(y_i \neq G(x_i))$$

– step (a) of the algorithm, with $\overline{\mathrm{err}}_m$ of Eq. (7.26). The second consequence is on the next slide.

Solving for β_m and recovering the weight update

Plug G_m into (7.25), divide by $\sum_i w_i^{(m)}$ and write $\overline{\mathrm{err}}_m$ for the weighted error:

$$h(\beta) = (e^{\beta} - e^{-\beta})\,\overline{\mathrm{err}}_m + e^{-\beta}.$$

$$h'(\beta) = (e^{\beta} + e^{-\beta})\,\overline{\mathrm{err}}_m - e^{-\beta} = 0$$

$$\Longrightarrow\quad e^{2\beta}\,\overline{\mathrm{err}}_m = 1 - \overline{\mathrm{err}}_m,$$

$$\beta_m = \tfrac{1}{2}\log\frac{1-\overline{\mathrm{err}}_m}{\overline{\mathrm{err}}_m}. \qquad (7.27)$$

The update of the model is $f_m = f_{m-1} + \beta_m G_m$, so the weights for the next stage are

$$w_i^{(m+1)} = e^{-y_i f_m(x_i)} = w_i^{(m)}\, e^{-\beta_m y_i G_m(x_i)}.$$

With $-y_i G_m(x_i) = 2I(y_i \neq G_m(x_i)) - 1$,

$$w_i^{(m+1)} = w_i^{(m)}\, e^{\alpha_m I(y_i \neq G_m(x_i))}\, e^{-\beta_m}, \qquad \alpha_m = 2\beta_m. \qquad (7.28,\ 7.29)$$

The factor $e^{-\beta_m}$ is common to all i and disappears in the normalization.

So the whole algorithm is one line

AdaBoost = forward stagewise additive modelling with the exponential loss, weak learner $G_m = \arg\min$ weighted error, step $\beta_m = \frac{1}{2}\log\frac{1-\overline{\mathrm{err}}_m}{\overline{\mathrm{err}}_m}$. The "strange" vote $\alpha_m = \log\frac{1-\overline{\mathrm{err}}_m}{\overline{\mathrm{err}}_m}$ is $2\beta_m$, the weight update is the loss evaluated at the new model.

Why the exponential loss?

It makes the stagewise problem solvable in closed form – (7.25) is linear in the indicator. Its *population* minimizer is

$$f^*(x) = \arg\min_f \mathbb{E}_{Y|x} e^{-Yf} = \tfrac{1}{2}\log\frac{P(Y=+1\mid x)}{P(Y=-1\mid x)},$$

half the log-odds (differentiate $pe^{-f} + (1-p)e^{f}$) – the same minimizer as the logistic loss, so AdaBoost estimates what logistic regression estimates, with trees instead of a linear function.

Exponential against logistic loss, and the margin

Both losses depend on the data only through the **margin** $yf(\mathrm{x})$: positive means correct, and $|yf|$ is the confidence.

loss	$L(yf)$	for $yf \to -\infty$
misclassification	$I(yf < 0)$	1
exponential (7.23)	e^{-yf}	$e^{\lvert yf\rvert}$
logistic (binomial)	$\log(1 + e^{-2yf})$	$2\lvert yf\rvert$
squared	$(1 - yf)^2$	$\lvert yf\rvert^2$

All but the first are convex, differentiable upper bounds of the misclassification loss, and all three keep pushing *after* the training error is zero – they reward larger margins. The exponential loss does it **exponentially hard**: a point with margin -3 costs $e^3 \approx 20$ times a point with margin 0.

Consequences you will see in the experiments

- AdaBoost's **training error keeps falling after it is already zero** in the margin sense: the distribution of $y_i f(\mathrm{x}_i)/\sum \alpha_m$ moves right. That is Schapire's margin explanation of why boosting overfits more slowly than one expects.
- On **noisy labels** the exponential loss is a liability: a mislabelled point gets weight $e^{|f|}$ and the stumps start chasing it. The logistic loss grows linearly and is far more robust – that is one reason gradient boosting replaced AdaBoost in practice.
- The squared loss is a poor classification loss: it *penalizes* margins larger than one, that is, points classified correctly with confidence.

Pause and think 2: what if the weak learner is too good?

Question

Suppose at round m the stump happens to get *every* weighted point right, $\overline{\mathrm{err}}_m = 0$. What does the algorithm do? And what does it do when a weak learner is *worse* than a coin, $\overline{\mathrm{err}}_m > 1/2$?

Pause and think 2: what if the weak learner is too good?

Question

Suppose at round m the stump happens to get *every* weighted point right, $\overline{\mathrm{err}}_m = 0$. What does the algorithm do? And what does it do when a weak learner is *worse* than a coin, $\overline{\mathrm{err}}_m > 1/2$?

Answer

$\overline{\mathrm{err}}_m = 0$ gives $\alpha_m = \infty$: the ensemble becomes G_m alone and the weights of the other points go to zero – the algorithm stops, correctly: it has found a perfect classifier and no further term can help (our code clips $\overline{\mathrm{err}}_m$ at 10^{-10} and breaks). For $\overline{\mathrm{err}}_m > 1/2$, $\alpha_m < 0$: the vote is *negative*, which is the same as using $-G_m$ with a positive vote, and $-G_m$ has error $1 - \overline{\mathrm{err}}_m < 1/2$. The weight update then *decreases* the weight of the misclassified points, again consistent with using $-G_m$. A stump learner never produces $\overline{\mathrm{err}}_m > 1/2$ because it tries both orientations, but a fixed-orientation learner could, and the formulas handle it. The only case that breaks the algorithm is $\overline{\mathrm{err}}_m = 1/2$ exactly: $\alpha_m = 0$, the weights do not change, and every later round repeats the same stump – which is why the implementations stop there.

Code 1: the only change the tree needs – sample weights

Last week's `DecisionTree` with weights w_i in the impurity, the leaf vote and the size of the children (Section 7.6):

```
def _impurity(self, y, w=None):
    if w is None: # last week's unweighted version
        return gini(y, self.classes_) if self.criterion == "gini" \
               else entropy(y, self.classes_)
    p = np.array([w[y == c].sum() / w.sum() for c in self.classes_]) # weighted
        fractions
    if self.criterion == "gini":
        return 1.0 - np.sum(p**2)
    p = p[p > 0]
    return -np.sum(p * np.log2(p))

def _leaf_value(self, y, w=None):
    counts = ([np.sum(y == c) for c in self.classes_] if w is None
              else [w[y == c].sum() for c in self.classes_])
    return self.classes_[int(np.argmax(counts))] # (weighted) majority vote

# inside _best_split: the children are weighed by their weight, not their size
wl, wr = w[mask].sum(), w[~mask].sum()
child = (wl / W) * self._impurity(y[mask], w[mask]) + (wr / W) * self._impurity(y[~
       mask], w[~mask])
gain = parent - child
```

`fit(X, y, sample_weight=w)` passes the weights down the recursion; everything else – thresholds at midpoints, the stopping rules, prediction – is unchanged.

Why weights and not resampling

The original papers resampled the data with probabilities w_i and fitted an ordinary tree; the weighted impurity is the deterministic version and is what `scikit-learn` does (`sample_weight`). No sampling noise – which is why our numbers will agree with the library to every digit.

Cost

A stump on n points and p features costs $O(pn \log n)$; M rounds cost M times that. Our pure-Python tree takes 2.4 s for 200 stumps on 375 points; `scikit-learn` a fraction of a second.

Code 2: the AdaBoost class of Section 7.12, and the library

```
class AdaBoost:
    """Discrete AdaBoost with stumps (or small trees); labels -1 and +1."""
    def __init__(self, n_estimators=50, max_depth=1):
        self.n_estimators, self.max_depth = n_estimators, max_depth

    def fit(self, X, y):
        n = len(y)
        w = np.full(n, 1.0 / n) # step 1: uniform weights
        self.trees_, self.alphas_, self.errors_ = [], [], []
        for _ in range(self.n_estimators):
            tree = DecisionTree("classification", "gini", max_depth=self.max_depth
                    )
            tree.fit(X, y, sample_weight=w) # (a) weighted weak learner
            miss = (tree.predict(X) != y).astype(float)
            err = np.sum(w * miss) / np.sum(w) # (b) Eq. (7.26)
            err = min(max(err, 1e-10), 1 - 1e-10)
            alpha = np.log((1 - err) / err) # (c) Eq. (7.29)
            w = w * np.exp(alpha * miss) # (d) Eq. (7.28), then renormalize
            w /= w.sum()
            self.trees_.append(tree); self.alphas_.append(alpha); self.errors_.
                    append(err)
            if err <= 1e-10:
                break
        return self

    def decision_function(self, X, M=None): # the ensemble after M rounds
        M = len(self.trees_) if M is None else M
        return np.sum([a * t.predict(X) for a, t in zip(self.alphas_[:M], self.
                trees_[:M])], axis=0)

    def predict(self, X, M=None):
        return np.sign(self.decision_function(X, M)) # Eq. (7.22)
```

Twenty lines, each one a line of the algorithm. The optional `M` evaluates the ensemble truncated after M rounds, so one fit gives the whole error curve – the library's `staged_predict`.

The same in `scikit-learn`

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
sk = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
        n_estimators=200, learning_rate=1.0,
        random_state=2026).fit(Xtr, ytr)
test_err = [np.mean(p != yte) for p in sk.staged_predict(Xte)]
sk.estimator_weights_ # our alphas_
sk.estimator_errors_ # our errors_
```

`learning_rate` multiplies every α_m (shrinkage); with 1.0 it is the algorithm on the left. In `scikit-learn` 1.9 the `algorithm` argument is gone: SAMME, the multi-class version, is the only one, and for two classes it *is* AdaBoost. Labels may be $\{0, 1\}$ there; our class wants ± 1.

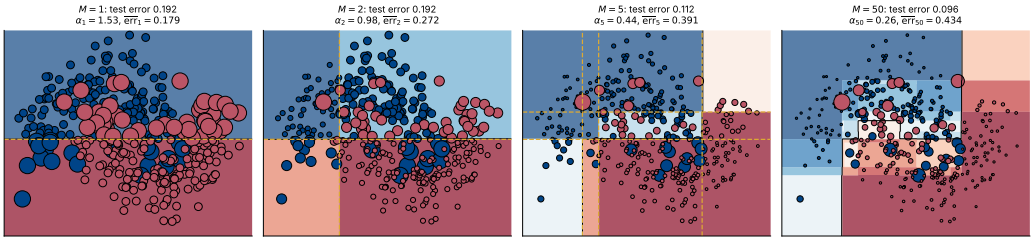
Ours against `scikit-learn` on the moons: identical to every digit

Moons, $n = 500$, noise 0.3, 375 training and 125 test points, seed 2026. Misclassification error after M rounds:

	stumps (depth 1)				trees of depth 3			
	ours		scikit-learn		ours		scikit-learn	
M	train	test	train	test	train	test	train	test
1	0.179	0.192	0.179	0.192	0.088	0.088	0.088	0.088
5	0.096	0.112	0.096	0.112	0.077	0.088	0.077	0.088
20	0.096	0.112	0.096	0.112	0.019	0.096	0.019	0.096
50	0.077	0.096	0.077	0.096	0.000	0.088	0.000	0.088
100	0.077	0.104	0.077	0.104	0.000	0.104	0.000	0.104
200	0.061	0.112	0.061	0.112	0.000	0.120	0.000	0.120

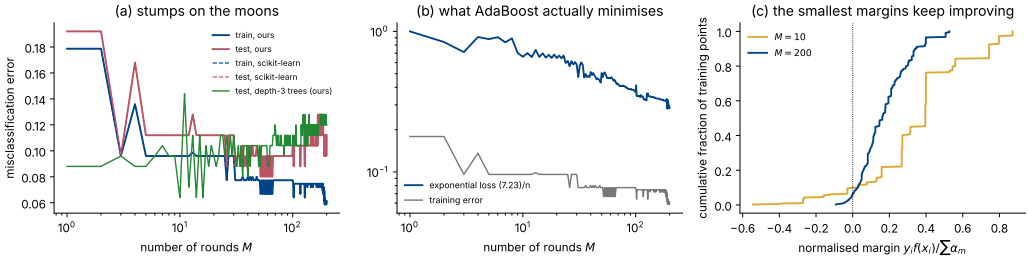
The first five votes agree too: $\alpha = (1.525, 0.984, 1.080, 0.847, 0.442)$ from errors $(0.179, 0.272, 0.254, 0.300, 0.391)$ – ours and `estimator_weights_` coincide, because both solve the same deterministic weighted problem. **Stumps**: the test error falls from 0.19 to about 0.10 and then wanders; a stump ensemble is additive in x_1 and x_2 and cannot carve the two crescents. **Depth 3**: zero training error at $M = 50$, and the test error then *rises* from 0.088 to 0.120 – the exponential loss chasing the noisy points. Remember that the Bayes error of these moons is about 0.08 and that the random forest reached 0.091 last week.

The boundary and the weights after 1, 2, 5 and 50 stumps



Colour: the sign and size of $f_M = \sum_{m \le M} \alpha_m G_m$; black line $f_M = 0$; the area of each training point is its weight w_i after round M; dashed lines are the stumps used so far. The first stump is horizontal ($x_2 \le 0.16 \Rightarrow$ class 0) and the tips of both crescents, which lie on the wrong side of it, take the weight; the second and third are vertical ($x_1 \le -0.39$, $x_1 \le 1.53$) and cut those tips off; by $M = 5$ the weights sit on the overlap region and the boundary is a staircase; by $M = 50$ it is the sum of fifty thresholds – jagged, but tracking the crescents.

Error, exponential loss and margins against the number of rounds



(a) ours (solid) and `scikit-learn` (dashed) lie on top of each other; green: depth-3 trees. **(b)** the quantity AdaBoost actually minimizes, $\frac{1}{n}\sum_i e^{-y_i f_M(x_i)}$, falls monotonically while the 0–1 training error moves in steps and plateaus: the algorithm is a coordinate descent on (7.23), not on the error. **(c)** the cumulative distribution of the normalized margins $y_i f(x_i)/\sum_m \alpha_m$: from $M = 10$ to $M = 200$ the *smallest* margin rises from -0.54 to -0.09 and the fraction with negative margin from 0.096 to 0.061, while the median decreases – the ensemble spends its later rounds on the hardest points, and the margin of the easy ones is traded away.

Pause and think 3: is AdaBoost immune to overfitting?

Question

The depth-3 ensemble reaches zero training error at $M = 50$, and its test error then rises from 0.088 to 0.120 by $M = 200$. The folklore says boosting "does not overfit". Who is right?

Pause and think 3: is AdaBoost immune to overfitting?

Question

The depth-3 ensemble reaches zero training error at $M = 50$, and its test error then rises from 0.088 to 0.120 by $M = 200$. The folklore says boosting "does not overfit". Who is right?

Answer

The data. The folklore comes from clean benchmarks where the test error keeps improving long after the training error is zero, and the margin argument explains why it *can*. But the moons have label noise (about 8% of the points are on the wrong side of the Bayes boundary), and the exponential loss gives a point with margin -3 twenty times the weight of a point with margin 0; the later trees are fitted almost entirely to those noisy points and carve islands around them – exactly the single-tree pathology we averaged away last week. Three remedies, all in use: a **smaller step** (`learning_rate`< 1, shrinkage), a **robust loss** (logistic, Huber – gradient boosting, next part), and **choosing M by validation** (early stopping) instead of "as large as you can afford", which was the right answer for bagging and is the wrong one here.

From AdaBoost to any loss: the gradient with respect to the predictions

Stagewise fitting with an arbitrary differentiable loss L has no closed form. Friedman's idea (2001): treat the n predicted values $\mathrm{f} = (f(\mathrm{x}_1), \ldots, f(\mathrm{x}_n))^T$ as the parameters and do **gradient descent on the training loss**

$$\mathcal{L}(\mathrm{f}) = \sum_{i=1}^{n} L(y_i, f(\mathrm{x}_i)).$$

The gradient at the current model f_{m-1} has components

$$g_{im} = \left[\frac{\partial L(y_i, f(\mathrm{x}_i))}{\partial f(\mathrm{x}_i)}\right]_{f=f_{m-1}}, \qquad (7.30)$$

and steepest descent with an exact line search is

$$\rho_m = \arg\min_{\rho} \mathcal{L}(\mathrm{f}_{m-1} - \rho\, \mathrm{g}_m), \qquad \mathrm{f}_m = \mathrm{f}_{m-1} - \rho_m \mathrm{g}_m.$$

Each step adds a "function" $-\rho_m \mathrm{g}_m$: this *is* an additive model, Eq. (7.19), with the negative gradient as the m-th basis function.

The squared error worked out

$L = \frac{1}{2}(y - f)^2$ gives $g_{im} = -(y_i - f_{m-1}(\mathrm{x}_i))$: the negative gradient **is the residual**. Start from $f_0 = 0$: $\mathrm{g}_1 = -\mathrm{y}$. Line search:

$$\rho_1 = \arg\min_{\rho} \tfrac{1}{2}\|\mathrm{y} - \rho\mathrm{y}\|^2 \;\Rightarrow\; \rho_1 = 1, \qquad \mathrm{f}_1 = \mathrm{y}.$$

(With $L = (y - f)^2$ the gradient is $-2(\mathrm{y} - \mathrm{f})$ and $\rho_1 = \frac{1}{2}$; same f_1.) **One step fits every training point exactly**, the training loss is zero, and $\rho_2 = 0$ – our code prints `rho_1 = 0.5000, training MSE 0.00e+00, rho_2 = 0.0000`.

And that is the problem

f_1 is a list of n numbers at n points. It says *nothing* about any other x – there is no function to evaluate on the test set – and it has interpolated the noise. Steepest descent in function space, taken literally, is the perfect overfitter with zero generalization.

The repair: take the step along a tree instead of along the gradient

We want a step that (i) points roughly along $-\mathrm{g}_m$ at the training points and (ii) is a *function* defined everywhere, from a class with limited capacity. So **fit a regression tree T_m to the negative gradient**:

$$T_m = \arg\min_T \sum_{i=1}^{n} \big(- g_{im} - T(\mathrm{x}_i) \big)^2,$$

the squared-error projection of the steepest-descent direction onto the set of trees with J leaves. Then step along T_m:

$$f_m(\mathrm{x}) = f_{m-1}(\mathrm{x}) + \nu\, \rho_m\, T_m(\mathrm{x}),$$

$$\rho_m = \arg\min_\rho \sum_i L\big(y_i, f_{m-1}(\mathrm{x}_i) + \rho\, T_m(\mathrm{x}_i)\big).$$

With a tree the line search is done **leaf by leaf**: the tree fixes the regions R_{jm}, and the value in each region is

$$\gamma_{jm} = \arg\min_\gamma \sum_{\mathrm{x}_i \in R_{jm}} L\big(y_i, f_{m-1}(\mathrm{x}_i) + \gamma\big),$$

J one-dimensional problems instead of one.

Gradient boosting: the algorithm

1. $f_0(\mathrm{x}) = \arg\min_\gamma \sum_i L(y_i, \gamma)$ – the best constant ($\bar{y}$ for the squared error, the log-odds for the logistic loss).
2. For $m = 1, \ldots, M$:
   (a) pseudo-residuals $u_{im} = -g_{im}$, Eq. (7.30);
   (b) fit a regression tree to $\{(\mathrm{x}_i, u_{im})\}$, giving regions R_{jm}, $j = 1, \ldots, J_m$;
   (c) leaf values γ_{jm} by the one-dimensional line search above;
   (d) $f_m(\mathrm{x}) = f_{m-1}(\mathrm{x}) + \nu \sum_j \gamma_{jm} I(\mathrm{x} \in R_{jm})$.
3. Output f_M.

For the squared error this is literally **"fit a tree to the residuals and add it"**, Eq. (7.31): the leaf values are mean residuals, so (b) and (c) coincide. $\nu \in (0, 1]$ is the **shrinkage**.

Pseudo-residuals for the losses you will use

task	loss $L(y, f)$	$u = -\partial L/\partial f$	leaf value γ_j
regression	$\frac{1}{2}(y-f)^2$	$y - f$ (7.31)	mean of u in the leaf
regression, robust	$\lvert y - f \rvert$	$\operatorname{sign}(y-f)$	median of $y - f$ in the leaf
regression, Huber	quadratic, linear beyond $\pm\delta$	$y - f$ clipped at $\pm\delta$	robust location
two classes, $y \in \{0, 1\}$	binomial deviance, $p = \sigma(f)$	$y - p$ (7.32)	$\sum u / \sum p(1-p)$ (Newton)
K classes	multinomial deviance, softmax	$y_k - p_k$, one tree per class	Newton per class

The logistic case, Eq. (7.32)

With $p = \sigma(f) = 1/(1+e^{-f})$ and $\partial p/\partial f = p(1-p)$,

$$-\frac{\partial L}{\partial f} = -\Big[-\frac{y}{p} + \frac{1-y}{1-p}\Big]p(1-p) = y - p,$$

the same "observed minus predicted" residual as in logistic regression and in a network's cross-entropy output layer. Gradient boosting fits trees to $y - p$ and adds them to the *log-odds*.

The leaf value is a Newton step

For the logistic loss the line search in a leaf has no closed form; one Newton step from $\gamma = 0$ gives

$$\gamma_j = \frac{\sum_{i \in R_j}(y_i - p_i)}{\sum_{i \in R_j} p_i(1-p_i)} = -\frac{\sum_i g_i}{\sum_i h_i},$$

first over second derivatives. Keep it: XGBoost is built on it, with a $+\lambda$ in the denominator.

Code 3: our own gradient-boosting regressor, and the library

Section 7.13, squared error, last week's regression tree on the residuals:

```
class GradientBoostingRegressor:
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3)
            :
        self.n_estimators, self.lr, self.max_depth = n_estimators,
                learning_rate, max_depth

    def fit(self, X, y):
        self.f0_ = y.mean() # the best constant
        f = np.full(len(y), self.f0_)
        self.trees_ = []
        for _ in range(self.n_estimators):
            residual = y - f # negative gradient, Eq. (7.31)
            tree = DecisionTree("regression", max_depth=self.max_depth).
                    fit(X, residual)
            f = f + self.lr * tree.predict(X) # shrunken step
            self.trees_.append(tree)
        return self

    def predict(self, X, M=None):
        f = np.full(X.shape[0], self.f0_)
        for tree in self.trees_[:M]:
            f = f + self.lr * tree.predict(X)
        return f
```

Depth 2, $\nu = 0.1$, $M = 100$: test MSE **ours** 0.0128, `scikit-learn` 0.0113, noise 0.0100. The library's trees use Friedman's improved MSE split criterion and break ties differently; the algorithm is the same.

The data of the DocOnce notes: $n = 100$ points, $x \in [-3, 3]$,

$$y = e^{-x^2} + \tfrac{3}{2} e^{-(x-2)^2} + \epsilon, \quad \epsilon \sim N(0, 0.1^2),$$

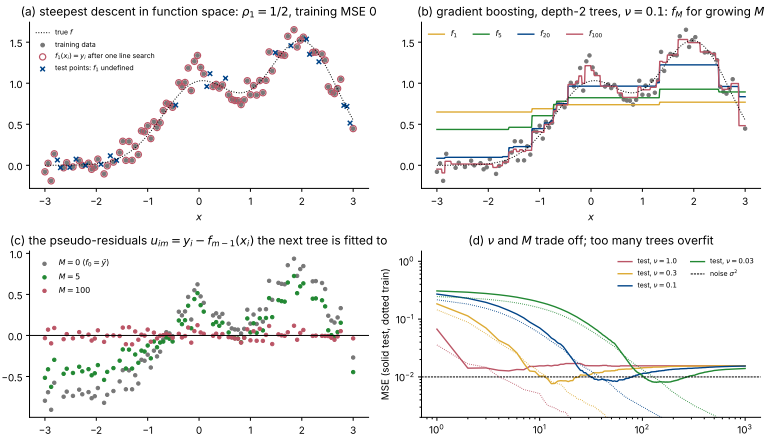
80 for training, 20 for testing.

```
from sklearn.ensemble import GradientBoostingRegressor
gb = GradientBoostingRegressor(max_depth=2, learning_rate=0.1,
        n_estimators=300, random_state=2026).fit(Xtr, ytr)
fits = list(gb.staged_predict(xgrid)) # f_1, f_2, ..., f_300 on a grid
test_mse = [np.mean((p - yte)**2) for p in gb.staged_predict(Xte)]
```

Arguments worth knowing

- `loss`: `squared_error`, `absolute_error`, `huber`, `quantile` – the table two slides back;
- `subsample`< 1: stochastic gradient boosting;
- `n_iter_no_change`, `validation_fraction`: early stopping;
- `max_depth` / `max_leaf_nodes`: the interaction order.

Steepest descent, staged fits, residuals and the ν–M trade-off



(a) literal steepest descent: after one line search $f_1(x_i) = y_i$ at every training point (red circles), and f_1 does not exist at the test points (blue crosses). **(b)** gradient boosting with depth-2 trees: f_1 is a two-split staircase, f_{100} follows the two bumps. **(c)** the pseudo-residuals $u_{im} = y_i - f_{m-1}(x_i)$ the next tree sees: structure at $M = 0$, structure at $M = 5$, noise at $M = 100$. **(d)** test MSE against M for four learning rates.

Shrinkage: ν and M are not independent knobs

Panel (d) in numbers (depth 2, up to 1000 trees):

ν	best M	best test	at M=1000
1.00	5	0.0127	0.0155
0.30	13	0.0076	0.0155
0.10	56	0.0084	0.0154
0.03	137	0.0080	0.0140

- Smaller ν needs proportionally more trees: $\nu M \approx$ constant at the optimum (5, 3.9, 5.6, 4.1).
- Smaller ν reaches a **lower minimum** and degrades more slowly past it; $\nu = 1$ overfits after five trees.
- Every curve eventually rises: unlike bagging, **more trees is not free**. The training MSE (dotted) goes to zero for $\nu = 1$ within a hundred trees – the ensemble has interpolated the noise.

Why small steps generalize better

Stagewise fitting with a small ν traces a *regularization path*: after M steps the model has moved a distance of order νM from the constant, in small increments that each take the direction of the current residual. Early stopping on this path behaves like the ℓ_1 / ℓ_2 penalties of week 36 – for a linear basis one can show the stagewise path with $\nu \to 0$ coincides with the lasso path (Efron et al. 2004). Large steps commit early to the first residual's shape, noise included; small steps keep correcting. The price is computation, which is why $\nu \in [0.01, 0.1]$ with M chosen by validation is the universal recipe.

The other knobs

Depth d (interaction order; $d = 2$–6 is typical, stumps for additive problems), **subsample** < 1 (a random half of the data per tree – variance reduction *inside* boosting, Friedman 2002), **min samples per leaf**, and the **loss**.

Pause and think 4: where did the bias go?

Question

Each depth-2 tree is a piecewise constant with at most four pieces – a terrible approximation to two smooth bumps. Yet f_{100} in panel (b) follows them closely. Where did the bias of the weak learner go, and why does the same not happen when we *bag* depth-2 trees?

Pause and think 4: where did the bias go?

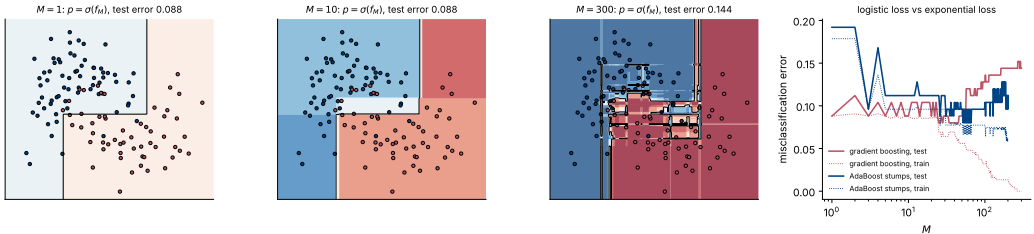
Question

Each depth-2 tree is a piecewise constant with at most four pieces – a terrible approximation to two smooth bumps. Yet f_{100} in panel (b) follows them closely. Where did the bias of the weak learner go, and why does the same not happen when we *bag* depth-2 trees?

Answer

Into the sum. Every tree is fitted to the *residual* of the previous ones, so its four pieces are placed where the current error is largest; a hundred trees give up to a few hundred distinct cutpoints, arranged adaptively – a piecewise constant with hundreds of pieces whose positions were chosen by the data. That is a rich class; the bias is small. Bagged depth-2 trees are all fitted to the *same* target, so their cutpoints cluster around the same few positions (the biggest drops of the function) and the average is a slightly smoothed four-piece staircase: the same bias as one tree, Eq. (7.16) in action. Boosting reduces bias sequentially; bagging reduces variance in parallel. (Boosting also reduces variance when ν is small and `subsample`< 1 – but that is a bonus, not the mechanism.)

Classification: the logistic loss on the moons, against AdaBoost



`GradientBoostingClassifier(max_depth=2, learning_rate=0.1)`: the colour is $p = \sigma(f_M)$, a *probability*, which AdaBoost's $\operatorname{sign} f$ never gave us. Test error 0.088 at $M = 1$ and $M = 10$, best 0.080 at $M = 25$ – the Bayes error – then 0.128 at $M = 100$ and 0.144 at $M = 300$ as the islands appear. Right: against AdaBoost stumps (blue). Both overfit eventually; gradient boosting with the logistic loss gets *lower* first, with depth-2 trees that can represent the crescents, and the cure is the same: stop early. Note the training error of gradient boosting reaching zero while the test error climbs.

XGBoost (Chen and Guestrin 2016): second-order boosting with regularized trees

Gradient boosting uses the first derivative to choose the direction and a line search for the step. XGBoost expands the loss at stage m to **second order** in the new tree $T(\mathrm{x})$ (Newton instead of steepest descent), and adds an explicit **penalty on the tree**:

$$\mathcal{L}^{(m)} \approx \sum_i \left[g_i\, T(\mathrm{x}_i) + \tfrac{1}{2} h_i\, T(\mathrm{x}_i)^2 \right] + \Omega(T), \tag{7.33}$$

$$g_i = \frac{\partial L(y_i, f)}{\partial f}\Big|_{f_{m-1}(\mathrm{x}_i)}, \qquad h_i = \frac{\partial^2 L(y_i, f)}{\partial f^2}\Big|_{f_{m-1}(\mathrm{x}_i)},$$

$$\Omega(T) = \gamma J + \tfrac{1}{2}\lambda \sum_{j=1}^{J} w_j^2, \tag{7.34}$$

for a tree with J leaves and leaf values w_j: γ charges for every leaf, λ is a ridge penalty on the leaf values; the constant $L(y_i, f_{m-1})$ was dropped.

g_i and h_i for the usual losses

loss	g_i	h_i
$\frac{1}{2}(y-f)^2$	$f_{m-1}(\mathrm{x}_i) - y_i$	1
logistic, $p = \sigma(f)$	$p_i - y_i$	$p_i(1-p_i)$
Poisson, $\mu = e^f$	$\mu_i - y_i$	μ_i

The same g and h as in the Newton leaf step of the previous part. Any loss with two derivatives works – including one you write yourself (Section 7.15).

Why second order

Newton's method has the step built in, $-g/h$, and a quadratic model is what makes leaf values and split score come out in closed form.

The leaf values and the score of a tree, in closed form

A tree is $T(\mathrm{x}) = w_{q(\mathrm{x})}$ with q the leaf index. Group the sum by leaf, $I_j = \{i : q(\mathrm{x}_i) = j\}$:

$$\mathcal{L}^{(m)} \approx \sum_{j=1}^{J} \Big[G_j w_j + \tfrac{1}{2}(H_j + \lambda) w_j^2 \Big] + \gamma J, \qquad (7.35)$$

$$G_j = \sum_{i \in I_j} g_i, \qquad H_j = \sum_{i \in I_j} h_i.$$

Each leaf is a separate quadratic in w_j; minimizing,

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \mathcal{L}^{(m)*} = -\frac{1}{2} \sum_{j=1}^{J} \frac{G_j^2}{H_j + \lambda} + \gamma J. \qquad (7.36)$$

Two things we gained

- the **leaf values in closed form** for *any* twice differentiable loss – no line search, no per-loss code;
- a **score for a tree structure** that does not depend on the leaf values, so it can be used to *grow* the tree.

Reading Eq. (7.36)

- $w_j^* = -G_j/(H_j + \lambda)$ is a **Newton step** per leaf, damped by λ: the sum of the gradients over the sum of the curvatures.
- The score $-\frac{1}{2}\sum_j G_j^2/(H_j + \lambda) + \gamma J$ is the best the tree structure can do; the smaller, the better. It plays the role of the impurity in CART.
- For the squared error ($h_i = 1$, $\lambda = 0$) the leaf value is the mean residual $-G_j/n_j$: gradient boosting as before.
- Every quantity is a **sum over the points in a leaf** – what makes histogram binning and parallel split finding possible.

The gain of a split

A split of leaf I into $I_L \cup I_R$ changes the score (7.36) by

$$\text{Gain} = \tfrac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\Big] - \gamma. \qquad (7.37)$$

Grow the tree greedily on this gain, as CART grows on the impurity decrease; a split with $\text{Gain} < 0$ is not made – γ is a **built-in pruning threshold**, and λ **shrinks** leaves with little curvature H_j toward zero.

Growing one tree on the gain

1. Compute g_i, h_i once per round (they depend on f_{m-1} only).
2. For every feature and candidate threshold (histogram bin edges), accumulate G_L, H_L in one sweep, $G_R = G - G_L$, $H_R = H - H_L$, evaluate (7.37).
3. Take the best split if its gain is positive; recurse to `max_depth`, with `min_child_weight` bounding H_L, H_R.
4. Leaf values from (7.36) times ν; add the tree, next round.

A leaf by hand (logistic loss)

Three points in a leaf, labels $y = (1, 0, 1)$, current probabilities $p = (0.6, 0.6, 0.6)$:

$g = p - y = (-0.4,\ 0.6,\ -0.4)$, $\quad G = -0.20$,

$h = p(1-p) = (0.24,\ 0.24,\ 0.24)$, $\quad H = 0.72$.

Unregularized Newton step:
$w^* = -G/H = 0.278$ (push the log-odds up: two of three are positive, $p = 0.6$ was too low).
With $\lambda = 1$:
$w^* = -G/(H+\lambda) = 0.20/1.72 = 0.116$, less than half – because $H = 0.72 < \lambda$, a leaf of three points has little curvature to back its estimate.
Score of the leaf: $-\frac{1}{2}G^2/(H+\lambda) = -0.0116$.
Compare λ with H_j, not with n_j: that is the meaning of `min_child_weight` (a lower bound on H_j) in the library.

Sanity check: with the squared error XGBoost is CART on the residuals

Check: squared error, $h_i = 1$, $\lambda = \gamma = 0$. Then $G_j = -\sum_{I_j} r_i$ with r_i the residual, $H_j = n_j$, and

$$\text{Gain} = \frac{1}{2}\Big[n_L \bar{r}_L^2 + n_R \bar{r}_R^2 - n\,\bar{r}^2\Big], \tag{7.38}$$

which is exactly the decrease in the residual sum of squares of CART's regression split, Eqs. (7.11)–(7.12) of last week (expand $\sum (r_i - \bar{r})^2 = \sum r_i^2 - n\bar{r}^2$). So XGBoost with the squared loss and no regularization *is* gradient boosting with CART trees and the mean residual in each leaf; everything else is the second-order treatment of other losses and the two penalties.

Multiply by ν

The actual update is $f_m = f_{m-1} + \nu\, T_m$ with ν (`eta`, `learning_rate`) typically 0.01–0.3; the leaf weights of Eq. (7.36) are the full Newton step.

What is new, then

- the second-order treatment of *other* losses – the logistic leaf is $-G/(H+\lambda)$ with no line search;
- λ shrinks the small leaves, γ prunes while growing;
- the same gain drives the split search for every loss, so the tree code is written once.

Pause and think 5: what does λ do to a leaf with a single point?

Question

Squared error, one training point in a leaf with residual r. What are G, H and w^* for $\lambda = 0$, 1, 10? What does the answer say about the role of λ in a deep tree versus a shallow one, and what is the analogous quantity in ridge regression?

Pause and think 5: what does λ do to a leaf with a single point?

Question

Squared error, one training point in a leaf with residual r. What are G, H and w^* for $\lambda = 0$, 1, 10? What does the answer say about the role of λ in a deep tree versus a shallow one, and what is the analogous quantity in ridge regression?

Answer

$g = -r$, $h = 1$, so $G = -r$, $H = 1$ and $w^* = r/(1+\lambda)$: the full residual for $\lambda = 0$, half of it for $\lambda = 1$, a tenth for $\lambda = 10$. A leaf with n_j points has $H_j = n_j$ and $w^* = \bar{r}_j\, n_j/(n_j + \lambda)$: the shrinkage factor $n_j/(n_j + \lambda)$ is negligible for large leaves and strong for small ones – λ acts almost entirely on the **small leaves of deep trees**, the ones that fit single noisy points. It is ridge regression's $\beta_j \to \beta_j\, d_j^2/(d_j^2 + \lambda)$ of week 36 with the singular value replaced by the leaf's curvature. On the cancer data below, raising λ from 0 to 10 *increased* the mean number of leaves per tree from 3 to 5: with shrunken leaves the probabilities stay away from 0 and 1, the h_i stay large, and more splits clear the `min_child_weight` and γ thresholds. The knobs interact.

What else is in the box

Statistics

- **Shrinkage** ν and **column subsampling** per tree or per split (`colsample_bytree`, `_bylevel`) – the random-forest trick inside boosting – and **row subsampling** (`subsample`).
- **Early stopping** on a validation set (`early_stopping_rounds`): choose M by the held-out loss.
- **Any loss** with a gradient and a Hessian: regression, logistic, multinomial, Poisson, Cox survival, ranking – or your own Python function returning (g, h); Section 7.15 derives g and h by automatic differentiation.
- **Missing values**: each split learns a default direction for them (sparsity-aware split finding).

Engineering (where the "X" comes from)

- **Histogram split finding**: features are binned into ≤ 256 quantile buckets, so a split costs $O(\text{bins})$ instead of $O(n \log n)$; the `hist` method is the default.
- **Parallel** over features and over bins; GPU training; out-of-core blocks for data larger than memory.
- `scikit-learn`'s `HistGradientBoostingClassifier / Regressor` is the same design (binning, second-order leaves, ℓ_2 penalty, early stopping) and is 4–10 times faster than its older `GradientBoosting*` siblings; LightGBM and CatBoost are the other two libraries of the family.

On tabular data of moderate size, with heterogeneous features and interactions, gradient-boosted trees have been the method to beat for a decade; that is why Project 3's paths B and C both list them.

Code 4: XGBoost – regression with early stopping, and the cancer data

```
import xgboost as xgb
reg = xgb.XGBRegressor(n_estimators=2000, learning_rate=0.05,
        max_depth=2,
                       reg_lambda=1.0, early_stopping_rounds=50,
                              random_state=2026)
reg.fit(Xtr, ytr, eval_set=[(Xte, yte)], verbose=False)
reg.best_iteration # 134
mse = np.mean((reg.predict(Xte) - yte)**2)
```

Stops at $M = 135$ (no improvement of the held-out loss for 50 rounds). With 20 test points the test MSE is noisy – XGBoost 0.0200, `scikit-learn` with the same M 0.0090 – so we also measure the error against the *noise-free* function on a grid: XGBoost 0.0054, `scikit-learn` 0.0045, both well below the noise variance 0.0100. Same method, different defaults (`reg_lambda=1`, `min_child_weight=1`, histogram bins).

Honesty note: stopping on the test set makes the test error optimistic; in a project, hold out a *third* set or cross-validate.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import (RandomForestClassifier,
    GradientBoostingClassifier, HistGradientBoostingClassifier)
X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state
        =2026,
                                      stratify=y)
models = {
  "logistic": make_pipeline(StandardScaler(), LogisticRegression(max_iter
          =5000)),
  "forest": RandomForestClassifier(200, random_state=2026),
  "GB": GradientBoostingClassifier(max_depth=3, learning_rate=0.1,
                                   n_estimators=200),
  "HistGB": HistGradientBoostingClassifier(max_depth=3, learning_rate=0.1,
                                           max_iter=200),
  "XGBoost": xgb.XGBClassifier(n_estimators=200, learning_rate=0.1,
          max_depth=3,
                               reg_lambda=1.0, gamma=0.0, eval_metric="
                                   logloss")}
for name, m in models.items():
    m.fit(Xtr, ytr)
    p = m.predict_proba(Xte)[:, 1]
    print(name, m.score(Xte, yte), roc_auc_score(yte, p), log_loss(yte, p))

# logloss curves: fit with eval_set=[(Xtr, ytr), (Xte, yte)], read m.
        evals_result()
# importances: m.get_booster().get_score(importance_type="gain")
```

Five models, one loop, one `predict_proba` interface – the point of `scikit-learn`'s API, which XGBoost implements.

The cancer data: five methods, three metrics

398 training, 171 test tumours, 30 features, seed 2026:

method	acc.	AUC	logloss	time
logistic regression	**0.965**	**0.992**	**0.093**	0.06 s
random forest, 200 trees	0.953	0.980	0.332	0.19 s
gradient boosting, $d = 3$	0.953	0.988	0.238	0.40 s
hist gradient boosting	0.959	0.985	0.229	0.11 s
XGBoost, $d = 3$	0.947	0.986	0.138	0.05 s

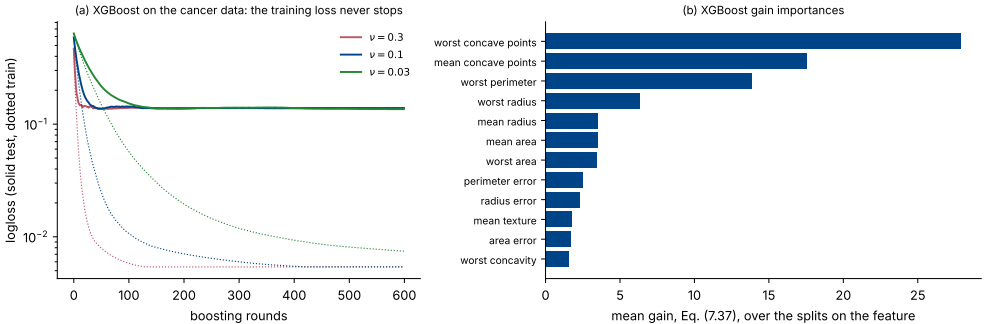
- Accuracies differ by **at most three tumours** out of 171; the ranking by accuracy is not meaningful at this n.
- The **logloss** separates them: the forest's vote fractions are poorly calibrated (0.33), the boosted log-odds much better, XGBoost's λ better still. AUC says all five rank the tumours almost equally well.
- XGBoost is the fastest of the ensembles – on 400 points. The engineering shows at $n = 4 \cdot 10^5$, not here.

The λ–γ scan (XGBoost, 200 trees, depth 3):

λ	γ	acc.	logloss	leaves
0	0	0.965	0.142	3
0	1	0.942	0.148	2
0	5	0.936	0.163	1
1	0	0.947	0.138	3
1	1	0.942	0.146	2
10	0	0.947	0.136	5
10	1	0.936	0.146	2
10	5	0.930	0.173	1

$\gamma = 5$ prunes every tree to a single leaf – a constant per round, the model degenerates to a slowly fitted intercept, and the error rises. $\gamma = 1$ leaves stumps. λ moves the logloss by a few thousandths and the accuracy by one or two tumours. On these data the defaults ($\lambda = 1$, $\gamma = 0$) are as good as anything; the scan is how you find out.

Logloss curves and importances



(a) test (solid) and training (dotted) logloss against the number of rounds for three learning rates. The training loss falls without end – with $\nu = 0.3$ to 10^{-3} – while the test loss flattens at 0.13–0.15 after 50–150 rounds and then creeps up: the picture to keep in mind whenever `n_estimators` is chosen by hand. **(b)** gain importances, Eq. (7.37) summed over the splits on each feature: the concave-points, perimeter and radius features that the random forest singled out last week, in a different order – importances are *model* properties, not data properties.

Pause and think 6: the simplest model won. Why, and what should you do about it?

Question

Logistic regression beats every tree ensemble on the cancer data on all three metrics. Why, and does that contradict the claim that boosted trees are the method to beat on tabular data?

Pause and think 6: the simplest model won. Why, and what should you do about it?

Question

Logistic regression beats every tree ensemble on the cancer data on all three metrics. Why, and does that contradict the claim that boosted trees are the method to beat on tabular data?

Answer

Because the problem is **nearly linearly separable** in the scaled features (the measurements are means and extremes of the same geometric quantities, strongly correlated, and the classes differ by a shift), the sample is small ($n = 398$, $p = 30$), and a linear model with 31 parameters has the right bias for exactly this situation – the trees spend their capacity reproducing a smooth oblique boundary with axis-aligned steps. No contradiction: the tabular-data claim is about data sets with thousands to millions of rows, heterogeneous features, interactions and missing values, where the linear model's bias dominates and the ensembles' flexibility pays. The lesson for Project 3 is procedural: **always fit the simple baseline first** and report it next to the complicated models; a method that does not beat logistic regression or OLS has not earned its complexity – and a method that does should be shown to do so on a held-out set, not on the set you tuned it on.

Bagging against boosting, in one table

	bagging / random forests (7.8–7.10)	boosting (7.11–7.15)
members	deep trees: low bias, high variance	shallow trees: high bias, low variance
built	independently, in parallel	sequentially, each on the residual of the rest
combined	average or majority vote, equal weights	weighted sum, β_m or α_m fitted
what it reduces	variance, down to the floor $\rho\sigma^2$, Eq. (7.16)	bias, stage by stage (and some variance)
the objective	none: a smoothing device	a loss, Eq. (7.20): exponential, logistic, squared, ...
number of trees B, M	as many as you can afford; never hurts	a tuning parameter; too many overfits
the main knobs	m (features per split), tree depth	ν, M, depth, subsample, λ, γ
free validation	out-of-bag error	early stopping on a held-out set
noisy labels	robust (averaging)	AdaBoost fragile, logistic / Huber loss robust
interpretability	importances, partial dependence	the same
typical use	a strong, almost tuning-free baseline	the strongest tabular model, when tuned

Both inherit the tree's properties – no extrapolation, axis-aligned boundaries, invariance to monotone feature transformations – and both lose the single tree's readability.

Boosting any loss by automatic differentiation (Section 7.15), in one slide

Everything in the XGBoost derivation needed only g_i and h_i: the first and second derivatives of $L(y_i, f)$ with respect to the scalar f at the current prediction. Section 7.15 computes them with the same automatic differentiation we used for the neural networks (JAX / PyTorch):

- write the loss as a Python function of f;
- `grad` and `grad(grad)` give g_i and h_i, vectorized with `vmap`;
- the leaf weights and the gain are Eqs. (7.36)–(7.37) – Eq. (7.38) in the squared-error case;
- grow the trees on that gain.

A quantile loss, a Poisson deviance, a physics-motivated asymmetric cost: no new derivation, just a new function. XGBoost's `objective` argument accepts such a function directly.

Why it matters for Project 3

Path C asks for the mathematics of the methods; a custom loss boosted with g and h from autograd, checked against XGBoost's built-in objective, is a complete and original small study. Path B can use it to put an asymmetric misclassification cost directly into the training objective instead of reweighting afterwards.

What the lecture adds up to

1. Ensembles exist because a tree has high variance; averaging cannot remove bias, Eq. (7.16) – week 46.
2. Boosting is forward stagewise fitting of an additive model, Eq. (7.20); the weak learner's depth sets the interaction order.
3. AdaBoost = exponential loss + stumps; the votes and weights follow from (7.23)–(7.29); it estimates half the log-odds. Ours matches `scikit-learn` to every digit.
4. Steepest descent in function space interpolates the training set in one step; restricting the step to a tree gives gradient boosting, with the residual (squared) or $y - p$ (logistic) as the target.
5. Shrinkage ν and M trade off; stop early.
6. XGBoost: second-order expansion, closed-form leaves $-G/(H + \lambda)$, gain with a γ threshold, histograms and parallelism.
7. Always report the simple baseline.

The last two weeks

This week

- **Tuesday November 17 – the final flipped classroom**: every group presents its **Project 3 proposal**, 3–5 slides, 3–5 minutes, with questions from the room. Which path (A: differential equations with neural networks, B: a classification problem with at least three methods, C: the statistics and mathematics of the methods), which data, which methods, which questions.
- **Weekly exercise for week 47** (the last one): the proposal slides, due **Friday November 20** at midnight on Canvas; bring the same slides on Tuesday. Include the LLM declaration.
- Lab sessions as usual: come with your proposal and your questions from today.

Week 48 – the last week

- **Monday November 23**: **summary lecture** of the whole course – from least squares to boosting, what connects the methods, how to choose among them, and what we did not cover.
- **No Tuesday session.**
- **Wednesday November 25, 12:00–16:00**: **lab only**, four hours of help with Project 3.

Project 3

Deadline **Friday December 11** at midnight. The report follows the Project 2 format; the updated LLM rules apply (text: level 1 only; code: declare the level). Today's material is directly usable in paths B and C: boosted trees as one of the three methods, or the bias–variance story of bagging against boosting as a topic.